Riesgo de los commits del kernel de Linux bajo deriva temporal: propuesta de un clasificador difuso-evolutivo interpretable y estado del arte

Linux kernel commit risk under temporal drift: an interpretable fuzzy-evolutionary classifier

Juan Felipe Rodríguez Galindo
Maestría en Ciencias de la Información y las Comunicaciones
Universidad Distrital Francisco José de Caldas, Bogotá, Colombia
Código 202615950004 · Inteligencia Computacional · Entregable 1

***Resumen*—Predecir si un commit del kernel de Linux requerirá corrección permite priorizar la revisión de código aprovechando el trailer `Fixes:`. Este trabajo formaliza el problema, la pregunta y el criterio de éxito del proyecto, y aporta una revisión estructurada tipo PRISMA (58 estudios incluidos, 37 citados aquí; la matriz completa va en el anexo). Se evidencia la falta de clasificadores de reglas difusas optimizados evolutivamente a nivel de commit bajo validación temporal y evaluados con AUC-PR, así como el estudio de su degradación ante la deriva en 2026.**

***Abstract*—Predicting Linux kernel commit risk via `Fixes:` trailers focuses code review. We define the problem, question, and success criteria, backed by a PRISMA review (58 included studies, 37 cited here; the full matrix is in the appendix). No prior study evaluates an evolutionarily tuned fuzzy rule classifier at commit level using temporal validation, AUC-PR, and drift analysis in 2026.**

***Palabras clave*—predicción de defectos justo a tiempo, kernel de Linux, deriva temporal, sistemas difusos, computación evolutiva, AUC-PR.**

I. INTRODUCCIÓN

Desde hace dos décadas, el riesgo de un commit se estima con métricas del propio cambio: cuánto toca, cuánto se difunde y la experiencia del autor [1], [2]. Es la predicción justo a tiempo (JIT): la alerta llega cuando el cambio aún está en revisión. El kernel de Linux cuenta con una ventaja clave: cuando un commit corrige un fallo previo, el trailer `Fixes:` identifica explícitamente al culpable sin recurrir a heurísticas como SZZ mediante `git blame` [3]. En estudios empíricos, el 81 % se confirmó como causa del bug; el resto es ambiguo o incorrecto [4].

En 2026 cambió quién escribe parte de estos cambios. La versión 7.0 pidió declarar con `Assisted-by:` la ayuda de asistentes de IA [5], simplificada luego a `Assisted-by: LLM` [6]. El 14 de octubre se retiró al constatar que solo cerca del 10 % de los commits la incluía, pese a un uso real mucho mayor [7]. Por ello, la «era de la IA» se define por frontera temporal (Fig. 1) y no por la etiqueta, que queda solo como referencia complementaria.

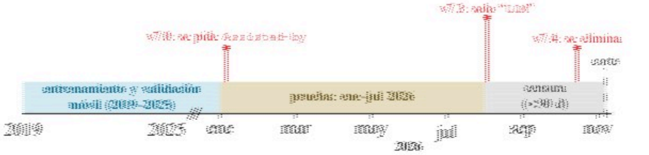

Figura 1. Diseño temporal propuesto frente a los hitos de `Assisted-by` [5]–[7]. Los commits posteriores a julio de 2026 quedan censurados: no han tenido los 90 días para recibir un `Fixes:`.

I-A. Problema y pregunta de investigación

Problema. Clasificar cada commit del kernel como *riesgoso* si un commit posterior lo cita con `Fixes:` en 90 días o menos, usando solo información disponible en el momento de integrarlo, y con un modelo que un mantenedor pueda leer.

Pregunta. ¿Un clasificador de a lo sumo 15 reglas difusas, cuyas funciones de pertenencia y reglas se ajustan con un algoritmo evolutivo, alcanza en los commits de 2026 un AUC-PR de al menos el 90 % del mejor modelo de referencia (regresión logística o *gradient boosting*), y pierde menos AUC-PR que ese modelo al pasar de la validación (ventanas anuales 2023, 2024 y 2025, entrenando desde 2019) a la prueba (enero–julio de 2026)?

I-B. Criterio de éxito (etapa 1 de CRISP-ML(Q) [8])

Sea AP_m^V el AUC-PR medio del modelo m en las ventanas de validación y AP_m^T su AUC-PR en la prueba de 2026, ambos con intervalo de confianza del 95 % por *bootstrap*. Sean $\Delta_m = (\mathrm{AP}_m^V - \mathrm{AP}_m^T)/\mathrm{AP}_m^V$ la caída relativa, π^T la prevalencia de positivos en la prueba (el AUC-PR de la clase mayoritaria), FE el modelo propuesto y R la mejor referencia clásica en validación. El proyecto se declara exitoso si se cumplen las tres condiciones de la Tabla I. Si no se cumplen, se reporta así, sin volver a ajustar nada con la prueba.

I-C. Datos y consideraciones éticas

Los datos son públicos: el historial oficial de `torvalds/linux` (GPL-2.0), minado directamente para un registro continuo de 2014 a 2026. Para contrastar la etiqueta se recurre además al dataset de Maes Bermejo *et al.*

Tabla I

CRITERIO DE ÉXITO, FIJADO ANTES DE VER LOS DATOS

	Condición
C1 Desempeño	$\mathrm{AP}^{T}_{FE} \geq 0{,}90\, \mathrm{AP}^{T}_{B}$
C2 Robustez	$\Delta_{FE} < \Delta_{B}$ (diferencia observada, con IC 95 %)
C3 Piso	El IC 95 % de AP^{T}_{FE} excluye π^{T}; McNemar contra cada referencia con $\alpha = 0{,}05$
Restricción	$\leq$15 reglas y $\leq$3 antecedentes por regla

en Zenodo [9]. Por ser fuentes enteramente abiertas no se requiere aprobación de un comité de ética; por privacidad, los nombres y correos de los autores se sustituyen por identificadores anónimos.

II. ESTADO DEL ARTE

II-A. Método de búsqueda

La búsqueda sigue las directrices de PRISMA 2020 [10] en versión sintetizada. El problema se descompone en tres componentes temáticos esenciales, unidos con OR por dentro y AND entre ellos:

A (commit* OR "code change*" OR "software change*" OR patch* OR "pull request*" OR "Linux kernel") AND
B ("defect prediction" OR "bug prediction" OR "bug-introducing" OR "bug-inducing" OR "fix-inducing" OR "just-in-time" OR "change risk" OR "risk prediction" OR "defect-prone" OR buggy) AND
C (fuzzy OR "rule-based" OR evolutionary OR "genetic algorithm" OR "genetic programming" OR "search-based" OR interpretab* OR explainab*)

La ecuación Q1 combina los tres bloques. La Q2 cambia el bloque C por deriva temporal ("concept drift", drift, "temporal validation", "time-aware", "realistic evaluation", "data shift", "distribution shift", "time travel"). Ambas consultas se ejecutaron sobre 2019–2026 vía APIs públicas en arXiv y OpenAlex (título y resumen) para garantizar reproducibilidad. OpenAlex indexa IEEE, ACM, Springer y Elsevier, supliendo la falta de acceso institucional a Scopus o IEEE Xplore. Clásicos previos a 2019 o hallados por citas entran como "otras fuentes".

Los criterios se definieron antes del tamizaje. Se incluye si: (I1) predice el riesgo de un cambio o estudia la validez de esa evaluación (orden temporal, latencia, ruido), (I2) reporta métricas cuantitativas, y (I3) describe sus datos. Se excluye si: (E1) trata generación o reparación de código, (E2) evalúa solo a nivel de archivo o módulo (se admitieron propuestas difusas y evolutivas de módulo ante la ausencia a nivel de commit), (E3) es de otro dominio, (E4) es solo un dataset sin modelo, o (E5) no se puede verificar. De las bases se evaluaron 47 registros en elegibilidad (10 excluidos y 37 incluidos); a estos se sumaron 21 de otras fuentes bajo los mismos criterios I y E, totalizando 58 estudios incluidos (37 citados aquí; la matriz completa va en el anexo). De ellos, 35 se leyeron a texto completo y 23 por resumen o fuentes secundarias debido a restricciones de acceso institucional. El flujo se resume en la Fig. 2.

II-B. Síntesis por enfoques

A. Métricas de cambio y aprendizaje clásico. La línea fundacional usa métricas del commit con regresión logística [1], [2]. Sigue siendo un punto de referencia muy sólido: con partición cronológica estricta, una regresión logística de una sola variable (líneas añadidas) iguala o supera en AUC a arquitecturas profundas como DeepJIT y CC2Vec [11].

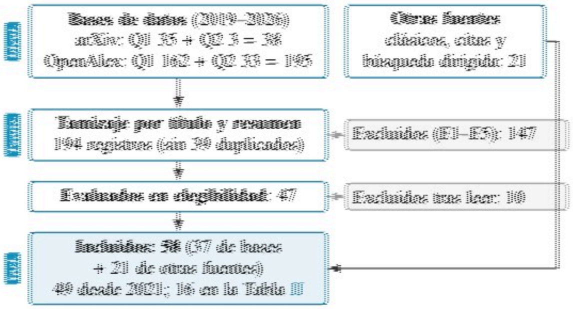
Bases de datos (2019–2026)
arXiv: Q1 35 + Q2 3 = 38
OpenAlex: Q1 162 + Q2 33 = 195
Otras fuentes
clásicos, citas y
búsqueda dirigida: 21
Tamizaje por título y resumen
194 registros (sin 39 duplicados)
Excluidos (E1–E5): 147
Evaluados en elegibilidad: 47
Excluidos tras leer: 10
Incluidos: 58 (37 de bases
+ 21 de otras fuentes)
49 desde 2021; 16 en la Tabla II

Figura 2. Flujo tipo PRISMA 2020 de la búsqueda. Conteos generados desde los registros de la búsqueda.

B. Aprendizaje profundo y LLM. Redes sobre el diff y modelos preentrenados con métricas [12] alcanzan techos de AUC-ROC cercanos a 0,88 en JIT-Defects4J. Tienen tres limitaciones clave: son cajas negras opacas, se evalúan casi exclusivamente en Java y rara vez reportan AUC-PR. En la industria, aunque se han usado LLMs para puntuar diffs, sus mapas de atención generaron baja confianza en los desarrolladores [13].

C. Interpretabilidad, difuso y evolutivo. Los profesionales piden explicaciones claras para cada alerta [25]. La respuesta habitual son explicadores *post hoc*: LIME en JITLine [14], reglas locales en PyExplainer [15] o commits afines en IRJIT [26], los cuales carecen de garantías formales de consistencia [27]. A nivel de commit, lo evolutivo y difuso se reducen a tareas accesorias: evolución diferencial para SMOTE [14], NSGA-II para calibrar parámetros logísticos [28], optimización de hiperparámetros [29], [30] o Fuzzy C-Means para modular capas de atención [31]. A nivel de módulo existen modelos neuro-difusos sobre datos NASA con particiones aleatorias y exactitud como métrica [16], [17] (donde una exactitud de 91 % enmascara una sensibilidad de apenas 0,33 [16]). No se identificó ningún clasificador de reglas difusas para riesgo de commits.

D. Evaluación realista, deriva y latencia. Los modelos JIT se degradan sustancialmente en el tiempo. En Qt y OpenStack, el AUC cae entre 11 y 34 puntos en solo un año sin reentrenar [18]. La validación cruzada infla los resultados: JITBoost cae de 0,891 a 0,863 de AUC al evaluar en partición temporal [32]. La deriva en JIT se aborda mediante detección continua [33], reciclaje de modelos [34], actualización en línea continua [20] o ventanas de espera [35]. Sin embargo, casi todos reportan AUC-ROC y omiten AUC-PR. La etiqueta llega tarde (latencia de 15 a más de 400 días [19]), lo que distorsiona las métricas hasta un 25 % si no se depura la cola temporal [36]. En el kernel de Linux este efecto es drástico: al pasar de datos balanceados artificialmente a la prevalencia real, el AUC-PR de ocho métodos cae de 0,805 a 0,016 [21]. Además, los commits de agentes no elevan la tasa de defectos, pero alteran

Tabla II

Matriz del estado del arte (16 de los 58 incluidos; la matriz completa va en el anexo). Enfoques: A métricas y aprendizaje clásico, B profundo y LLM, C interpretable, difuso o evolutivo, D evaluación realista y deriva, E etiqueta. †Solo nivel de módulo. ‡Preprint.

Referencia	Problema y datos	Paradigma y modelo	Referencias comparadas	Validación	Métrica y resultado	Limitaciones	Relevancia para el proyecto
A Kamei [2] (2013)	11 proyectos (6 libres, 5 comerciales), :≈250 mil cambios; SZZ	Reg. logística, 14 métricas de cambio	Azar	VC aleatoria 10 pliegues	Precisión 34 %, sensib. 64 %; AUC 0,73 (comerciales)	VC mezcla pasado y futuro	Variables base y referencia logística
B Zeng [10] (2021)	300 mil commits, 6 proyectos (Qt, OpenStack, Go…)	Réplica de DeepJIT y CC2Vec, LApredict (RL con una variable)	DeepJIT, CC2Vec, DBN-JIT, LR-JIT	Cronológica 80/20; recorta la cola	AUC media: LApredict 0,725; DeepJIT 0,707	Una corrida; solo AUC	Lo simple iguala a lo profundo con partición temporal
B Si [12] (2022)	JIT-Defects4J: 27 319 commits Java, 8,5 % positivos	JIT-Fine: CodeBERT + 14 métricas	LApredict, DeepJIT, CC2Vec, JITLine	Temporal 80/20 por proyecto	F1 0,431; AUC 0,881 (JITLine 0,264/0,802)	Solo Java, costo no reportado	Cota alta de AUC; modelo opaco
B Lin [13] (2026)	Diffs de Meta, cambios que causaron caídas (~2 por mil)	LLM (Diff Risk Score) + atención por bloque	Ninguna cuantitativa	No descrita	Top-2 bloques cubren 53,9 % de líneas causantes	Una empresa, 48 de 77 opiniones negativas	Explicación post hoc frente a reglas intrínsecas
C JITLine [14] (2021)	Qt y OpenStack, 37 524 commits	RF + tokens, SMOTE ajustado por evolución diferencial; LIME	EALR, DeepJIT, CC2Vec	Una partición temporal	AUC 0,83/0,82, F1 0,33/0,24	Una partición, sin pruebas estadísticas	Usa evolución y explica, pero sobre caja negra
C PyExplainer [15] (2021)	Qt y OpenStack, 40 798 commits	Reglas locales (RuleFit) sobre RF y RL	LIME	Temporal 70/30	RF: AUC 0,75/0,74; reglas 18–38 % más fieles que LIME	Dos proyectos; vecinos aleatorios	Reglas post hoc, no un modelo de reglas
C Aftab [16] (2023)†	NASA, 5 conjuntos, 3 579 módulos	Inferencia difusa (8 reglas) que combina NB, RNA y AD	NB, RNA, AD	Aleatoria 70/30	Exactitud 91,05 %, sensib. 0,33	Reglas a mano, exactitud con desbalance	Muestra por qué no usar exactitud
C Dey [17] (2025)†	NASA: JM1, KC1, PC1, KC2, CM1	Selección multiobjetivo (estrella de mar) + integral difusa de Choquet	7 clasificadores, MOGA, MOGWO	VC aleatoria 10 pliegues	Exactitud 0,85–0,96, sin AUC	Nivel módulo, F1 y sensib. casi perfectas con 7–20 % positivos	Lo difuso-evolutivo reciente no llega al commit
D McIntosh [18] (2018)	Qt y OpenStack, 37 524 cambios (8 y 13 %)	RL no lineal con splines	Periodo corto frente a largo	Longitudinal en el tiempo	AUC cae 11–34 puntos en un año; reentrenar recupera 6–24	Dos sistemas; filtros de SZZ	Evidencia de deriva; diseño a replicar
D Cabral [19] (2019)	10 proyectos GitHub, 2003–2018	En línea: ORB (árboles de Hoeffding)	OOB, UOB, ventana deslizante	Frecuencial con espera de 90 días	G-mean de JIT fija hasta 97,2 % menor que ORB	G-mean, sin AUC-PR	Sustenta la ventana de 90 días
D Olewicki [20] (2024)	Ubisoft: 4 proyectos de riesgo, 30–82 mil commits	MLP con aprendizaje continuo	XGBoost reentrenado, ingenuo	Secuencial en el tiempo	F1 +9 a +30 % en 6 de 8 proyectos	Una empresa; sin latencia	Deriva industrial en riesgo de commits
D Nguyen [21] (2025)	Linux (998 704 commits) y FFmpeg, Linux con `Fixes:`	8 técnicas JIT de vulnerabilidades (RL, DeepJIT, JIT-Fine…)	Escenario idealizado frente a realista	Temporal 75/5/20	PR-AUC en Linux 0,805 → 0,016 (−98 %)	Vulnerabilidades, no defectos	El más cercano: Linux, `Fixes:`, AUC-PR
D Mashoom-sadeh [22] (2026)‡	AIDev: 489 repos. Python, 9428 PR de agentes	Estudio longitudinal (GLMM, SZZ)	PR humanos	Por trimestres	Sin diferencia relevante en inducción de bugs (V = 0,008)	SZZ con P=0,39	La era IA cambia las covariables
E Lyu [4] (2024)	KFC: 76 046 commits con `Fixes:`, Linux 2013–2022	6 variantes de SZZ + TC-SZZ	Rosa et al.	Contra `Fixes:` como verdad	B-SZZ F1 0,51; 81 % de `Fixes:` confirmados	`Fixes:` es opcional	Valida la etiqueta del proyecto
E Herbold [23] (2022)	398 versiones, 38 proyectos Apache	Calidad de datos; NB entre proyectos	SZZ a 6 meses frente a etiqueta validada	Entre proyectos; Friedman + Nemenyi	Cerca de la mitad de las correcciones de SZZ no lo son	Solo Apache	Ruido de SZZ; preferir `Fixes:`
E Falessi [24] (2022)	19 proyectos Apache, :≈4000 defectos	15 clasificadores de Weka	Con y sin defectos latentes	Por versiones, 60/33	Pérdida mediana de sensib. ≈90 % por defectos latentes	Nivel de clase	Censura: recortar la cola reciente

el tamaño y estructura de los cambios [22], ubicando la deriva en las covariables.

E. Fiabilidad de la etiqueta. SZZ sufre de alto ruido: casi la mitad de los commits que señala como correctores no lo son [23], y el filtrado con LLM reduce 21 % los casos sospechosos en PostgreSQL [37]. En Linux, variantes de SZZ alcanzan un F1 de apenas 0,49–0,51 frente a la verdad del trailer `Fixes:` [4]. Defectos latentes aún no descubiertos bajan la sensibilidad cerca de un 90 % si no se aplica una ventana prudencial de censura temporal [24].

II-C. Brecha

Los enfoques anteriores dejan cuatro vacíos concretos:

1) Inexistencia de un clasificador de reglas difusas optimizado mediante algoritmos evolutivos para predecir riesgo a nivel de commit.
2) Escasa medición de la deriva temporal con AUC-PR y prevalencia real en commits del kernel respaldados por trailers `Fixes:`.
3) Falta de estudios sobre si la interpretabilidad intrínseca penaliza el desempeño o aporta robustez frente a la deriva.
4) 2026 es el primer año en que el kernel pidió declarar la ayuda de IA [5], aunque la etiqueta se retiró en octubre por su bajo uso [7].

ANEXOS

Comming soon 😖

DECLARACIÓN DE USO DE IA GENERATIVA

Usé Gemini para revisar y corregir errores y para redactar este informe a partir de los resultados del cuaderno y la recuperación de algunos papers para la evaluación del mismo.

REFERENCIAS

[1] A. Mockus and D. M. Weiss, "Predicting risk of software changes," Bell Labs Tech. J., vol. 5, no. 2, pp. 169–180, Apr.–Jun. 2000, doi: 10.1002/bltj.2229.

[2] Y. Kamei, E. Shihab, B. Adams, A. E. Hassan, A. Mockus, A. Sinha, and N. Ubayashi, "A large-scale empirical study of just-in-time quality assurance," IEEE Trans. Softw. Eng., vol. 39, no. 6, pp. 757–773, Jun. 2013, doi: 10.1109/TSE.2012.70.

[3] J. Śliwerski, T. Zimmermann, and A. Zeller, "When do changes induce fixes?" in Proc. Int. Workshop Mining Softw. Repositories (MSR '05), St. Louis, MO, USA, 2005, pp. 1–5, doi: 10.1145/1083142.1083147.

[4] Y. Lyu, H. J. Kang, R. Widyasari, J. Lawall, and D. Lo, "Evaluating SZZ implementations: An empirical study on the Linux kernel," IEEE Trans. Softw. Eng., vol. 50, no. 9, pp. 2219–2239, Sep. 2024, doi: 10.1109/TSE.2024.3406718.

[5] S. Levin, "docs: add AI Coding Assistants documentation," Linux kernel, commit 78d979db6cef, 6 ene. 2026. [En línea]. Disponible: https://github.com/torvalds/linux/commit/78d979db6cef

[6] C. Brauner, "coding-assistants: simplify attribution," Linux kernel, commit 816d9992d9ed, 3 ago. 2026. [En línea]. Disponible: https://github.com/torvalds/linux/commit/816d9992d9ed

[7] S. Levin, "docs: remove Assisted-by references," Linux kernel, commit 1e035297ffda, 14 oct. 2026. [En línea]. Disponible: https://github.com/torvalds/linux/commit/1e035297ffda

[8] S. Studer, T. B. Bui, C. Drescher, A. Hanuschkin, L. Winkler, S. Peters, and K.-R. Müller, “Towards CRISP-ML(Q): A machine learning process model with quality assurance methodology,” *Mach. Learn. Knowl. Extr.*, vol. 3, no. 2, pp. 392–413, 2021, doi: 10.3390/make3020020.

[9] M. Maes Bermejo, J. M. Gonzalez-Barahona, M. Gallego, and G. Robles, “A dataset of Linux kernel commits” (v3), Zenodo, 2024, doi: 10.5281/zenodo.10654193.

[10] M. J. Page *et al.*, “The PRISMA 2020 statement: An updated guideline for reporting systematic reviews,” *BMJ*, vol. 372, Art. no. n71, 2021, doi: 10.1136/bmj.n71.

[11] Z. Zeng, Y. Zhang, H. Zhang, and L. Zhang, “Deep just-in-time defect prediction: How far are we?,” in Proc. 30th ACM SIGSOFT Int. Symp. Softw. Testing Anal. (ISSTA), Virtual, Denmark, 2021, pp. 427-438, doi: 10.1145/3460319.3464819.

[12] C. Ni, W. Wang, K. Yang, X. Xia, K. Liu, and D. Lo, “The best of both worlds: Integrating semantic features with expert features for defect prediction and localization,” in Proc. 30th ACM Joint Eur. Softw. Eng. Conf. Symp. Found. Softw. Eng. (ESEC/FSE), Singapore, 2022, pp. 672-683, doi: 10.1145/3540250.3549165.

[13] Y. Liu, K. Jabre, R. Abreu, Z. J. Carmichael, V. Murali, A. Patel, J. Ge, W. Sun, C. Zhang, A. Mockus, D. Khavari, P. C. Rigby, and N. Nagappan, “A preliminary study on explaining risk of code changes using LLM-based prediction models,” in Proc. 3rd ACM Int. Conf. AI-Powered Software (AIware ’26), Montreal, QC, Canada, 2026, pp. 259–267, doi: 10.1145/3805760.3814916.

[14] C. Pornprasit and C. K. Tantithamthavorn, “JITLine: A simpler, better, faster, finer-grained just-in-time defect prediction,” in Proc. IEEE/ACM 18th Int. Conf. Mining Softw. Repositories (MSR), 2021, pp. 369-379, doi: 10.1109/MSR52588.2021.00049.

[15] C. Pornprasit, C. Tantithamthavorn, J. Jiarpakdee, M. Fu, and P. Thongtanunam, “PyExplainer: Explaining the predictions of just-in-time defect models,” in Proc. 36th IEEE/ACM Int. Conf. Automated Software Engineering (ASE), 2021, pp. 407–418, doi: 10.1109/ASE51524.2021.9678763.

[16] S. Aftab, S. Abbas, T. M. Ghazal, M. Ahmad, H. A. Hamadi, C. Y. Yeun, and M. A. Khan, “A cloud-based software defect prediction system using data and decision-level machine learning fusion,” Mathematics, vol. 11, no. 3, Art. no. 632, 2023, doi: 10.3390/math11030632.

[17] R. Dey, J. Piri, B. Acharya, P. P. Das, V. C. Gerogiannis, and A. Kanavos, “A hybrid evolutionary fuzzy ensemble approach for accurate software defect prediction,” Mathematics, vol. 13, no. 7, Art. no. 1140, 2025, doi: 10.3390/math13071140.

[18] S. McIntosh and Y. Kamei, “Are fix-inducing changes a moving target? A longitudinal case study of just-in-time defect prediction,” IEEE Trans. Softw. Eng., vol. 44, no. 5, pp. 412–428, May 2018, doi: 10.1109/TSE.2017.2693980.

[19] G. G. Cabral, L. L. Minku, E. Shihab, and S. Mujahid, “Class imbalance evolution and verification latency in just-in-time software defect prediction,” in Proc. IEEE/ACM 41st Int. Conf. Softw. Eng. (ICSE), Montreal, QC, Canada, 2019, pp. 666–676, doi: 10.1109/ICSE.2019.00076.

[20] D. Olewicki, S. Habchi, M. Nayrolles, M. Faramarzi, S. Chandar, and B. Adams, “On the costs and benefits of adopting lifelong learning for software analytics – Empirical study on brown build and risk prediction,” in Proc. 46th Int. Conf. Software Engineering: Software Engineering in Practice (ICSE-SEIP), Lisbon, Portugal, 2024, pp. 275–286, doi: 10.1145/3639477.3639717.

[21] D. Nguyen, T. Le-Cong, T. H. M. Le, M. A. Babar, and Q.-T. Huynh, “Toward realistic evaluations of just-in-time vulnerability prediction,” in Proc. IEEE Int. Conf. Software Maintenance and Evolution (ICSME), Auckland, New Zealand, 2025, pp. 1–10, doi: 10.1109/ICSME64153.2025.00023.

[22] I. Mazloomzadeh, M. M. Morovati, and F. Khomh, “How do AI coding agents contribute to software development? An empirical study of agentic pull requests,” arXiv:2607.21832, Jul. 2026, doi: 10.48550/arXiv.2607.21832.

[23] S. Herbold, A. Trautsch, F. Trautsch, and B. Ledel, “Problems with SZZ and features: An empirical study of the state of practice of defect prediction data collection,” Empir. Softw. Eng., vol. 27, no. 2, Art. no. 42, 2022, doi: 10.1007/s10664-021-10092-4.

[24] D. Falessi, A. Ahluwalia, and M. Di Penta, “The impact of dormant defects on defect prediction: A study of 19 Apache projects,” ACM Trans. Softw. Eng. Methodol., vol. 31, no. 1, pp. 1–26, Jan. 2022, doi: 10.1145/3467895.

[25] J. Jiarpakdee, C. K. Tantithamthavorn, and J. Grundy, “Practitioners’ perceptions of the goals and visual explanations of defect prediction models,” in Proc. IEEE/ACM 18th Int. Conf. Mining Software Repositories (MSR), 2021, pp. 432–443, doi: 10.1109/MSR52588.2021.00055.

[26] H. Sahar, A. A. Bangash, A. Hindle, and D. Barbosa, “IRJIT: A simple, online, information retrieval approach for just-in-time software defect prediction,” Empirical Softw. Eng., vol. 29, no. 5, Art. no. 131, 2024, doi: 10.1007/s10664-024-10514-z.

[27] J. Yu, M. Fu, A. Ignatiev, C. Tantithamthavorn, and P. Stuckey, “A formal explainer for just-in-time defect predictions,” ACM Trans. Softw. Eng. Methodol., vol. 33, no. 7, pp. 1–31, Aug. 2024, doi: 10.1145/3664809.

[28] X. Chen, Y. Zhao, Q. Wang, and Z. Yuan, “MULTI: Multi-objective effort-aware just-in-time software defect prediction,” Inf. Softw. Technol., vol. 93, pp. 1–13, Jan. 2018, doi: 10.1016/j.infsof.2017.08.004.

[29] J. Kang, S. Kwon, D. Ryu, and J. Baik, “HASPO: Harmony search-based parameter optimization for just-in-time software defect prediction in maritime software,” Appl. Sci., vol. 11, no. 5, Art. no. 2002, 2021, doi: 10.3390/app11052002.

[30] S. Mostafa, S. T. Cynthia, B. Roy, and D. Mondal, “Feature transformation for improved software bug detection and commit classification,” J. Syst. Softw., vol. 219, Art. no. 112205, Jan. 2025, doi: 10.1016/j.jss.2024.112205.

[31] F. Chen, Y. Ke, X. Liu, and Q. Wei, “Multi-project just-in-time software defect prediction based on multi-task learning for mobile applications,” in Proc. IEEE Conf. Software Testing, Verification and Validation (ICST), 2025, pp. 127–137, doi: 10.1109/ICST62969.2025.10989004.

[32] M. A. Shehab, A. Hamou-Lhadj, and V. S. P. Gunda, “JITBoost: Boosting just-in-time defect prediction using Boolean combination of classifiers,” in Proc. IEEE 23rd Int. Conf. Software Quality, Reliability and Security (QRS), 2023, pp. 95–104, doi: 10.1109/QRS60937.2023.00019.

[33] Z. Chitsazian, S. Sedighian Kashi, and A. Nikanjam, “Detecting concept drift in just-in-time software defect prediction using model interpretation,” Int. J. Syst. Assur. Eng. Manag., vol. 17, no. 9, pp. 2701-2722, 2026, doi: 10.1007/s13198-025-03122-7.

[34] H. J. Patel, B. Adams, and A. E. Hassan, “Post deployment recycling of machine learning models,” Empirical Softw. Eng., vol. 29, no. 4, Art. no. 100, 2024, doi: 10.1007/s10664-024-10492-2.

[35] D. Tian, L. Song, and X. Su, “Just-in-time quality assurance with assistance of automated defect prediction and code testing,” ACM Trans. Softw. Eng. Methodol., vol. 35, no. 9, pp. 1–46, 2026, doi: 10.1145/3779653.

[36] L. Song and L. L. Minku, “A procedure to continuously evaluate predictive performance of just-in-time software defect prediction models during software development,” IEEE Trans. Softw. Eng., vol. 49, no. 2, pp. 646–666, Feb. 2023, doi: 10.1109/TSE.2022.3158831.

[37] R. C. Veras, G. G. Cabral, and A. L. I. de Oliveira, “Semantic SZZ: Mitigating the impact of misclassified corrective changes in just-in-time software defect prediction,” in Proc. IEEE Int. Conf. Syst., Man, Cybern. (SMC), 2025, pp. 769-774, doi: 10.1109/SMC58881.2025.11342537.